

IEE3584 – Comunicaciones Inalámbricas

Tarea 14

Nicolás Seguel

Pregunta 32: Decodificación ZF

En esta pregunta se realizan simulaciones Monte Carlo para estimar el BER promedio utilizando decodificación **Zero-Forcing (ZF)** en configuraciones MIMO con modulación QPSK sobre un canal Rayleigh plano.

32.a) Simulación ZF para configuraciones 1x1, 2x1, 4x1 y 8x1

En este inciso se simulan sistemas MISO con un receptor y distintos números de antenas transmisoras. Las figuras siguientes muestran el BER simulado para cada configuración junto a las curvas teóricas 1x1.

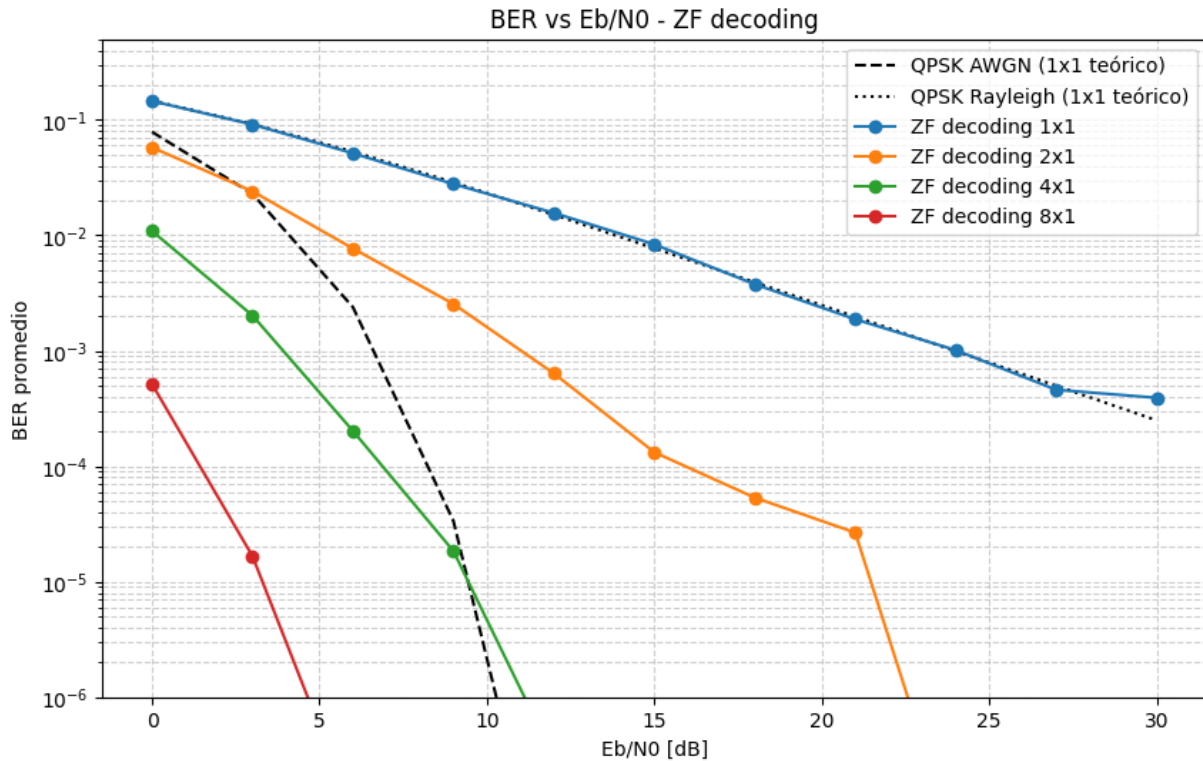


Figura 1: BER vs E_b/N_0 para configuraciones ZF: 1x1, 2x1, 4x1 y 8x1.

A mayor número de antenas transmisoras, se aprecia una mejora significativa en la pendiente de decaimiento del BER, especialmente en 4x1 y 8x1. Esto es esperable debido a la ganancia de diversidad en el enlace ascendente. La curva 1x1 coincide con el desempeño teórico en AWGN, validando la implementación.

32.b) Simulación ZF para configuraciones 1x1, 2x2, 4x4 y 8x8

Ahora se evalúan configuraciones cuadradas MIMO con el mismo número de antenas transmisoras y receptoras.

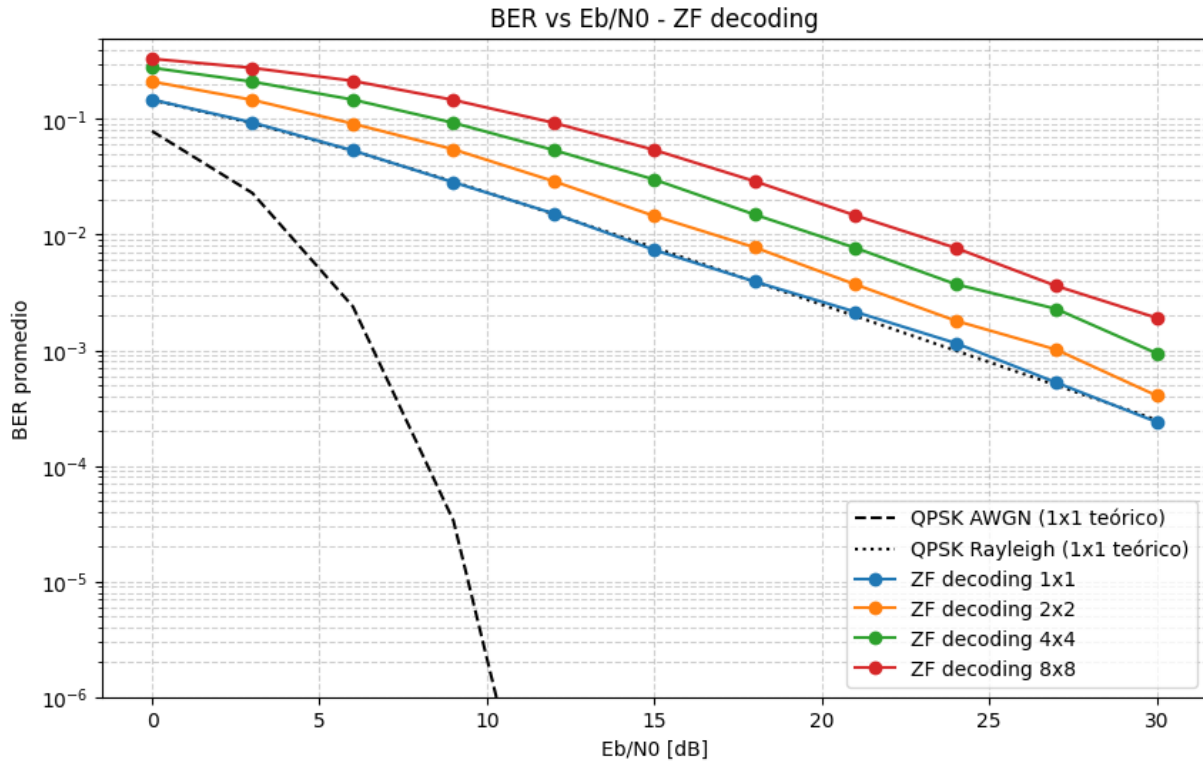


Figura 2: BER vs E_b/N_0 para configuraciones ZF: 1x1, 2x2, 4x4 y 8x8.

El desempeño decrece al aumentar el tamaño de la matriz MIMO, debido a que la energía se reparte entre todas las transmisiones simultáneas. Además, la amplificación del ruido inherente al ZF se vuelve más pronunciada en matrices más grandes, afectando negativamente el BER.

32.c) Simulación ZF para configuraciones 2x1, 4x2 y 8x4

Se simulan ahora sistemas MIMO sobredimensionados ($N_r > N_t$), en los cuales ZF opera en condiciones particularmente favorables.

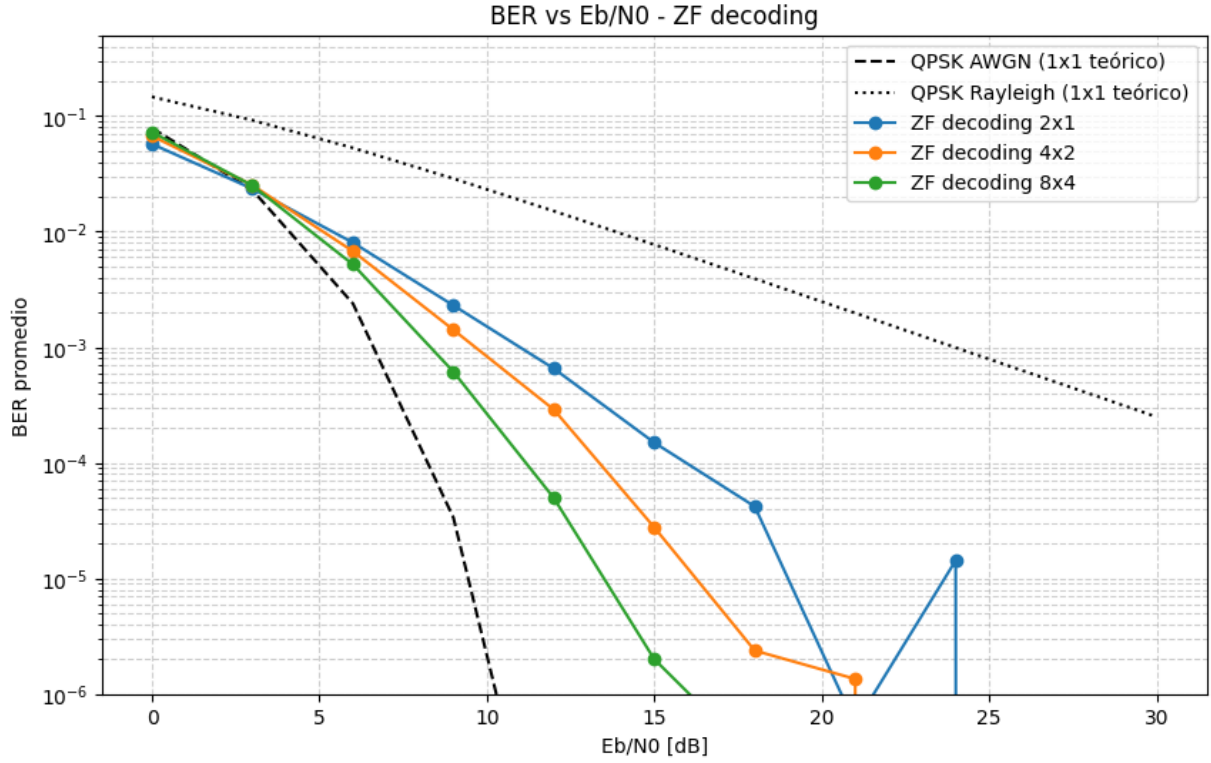


Figura 3: BER vs E_b/N_0 para configuraciones ZF: 2×1 , 4×2 y 8×4 .

Las tres configuraciones presentan mejoras notables respecto al caso cuadrado. Tener más antenas receptoras que transmisoras aumenta la dimensionalidad del canal y reduce sustancialmente la amplificación del ruido típica del ZF.

Pregunta 33: Decodificación MMSE

Ahora se repiten exactamente las mismas configuraciones anteriores, pero utilizando **MMSE**, el cual agrega un término de regularización para mitigar la amplificación de ruido.

33.a) Simulación MMSE para configuraciones 1x1, 2x1, 4x1 y 8x1

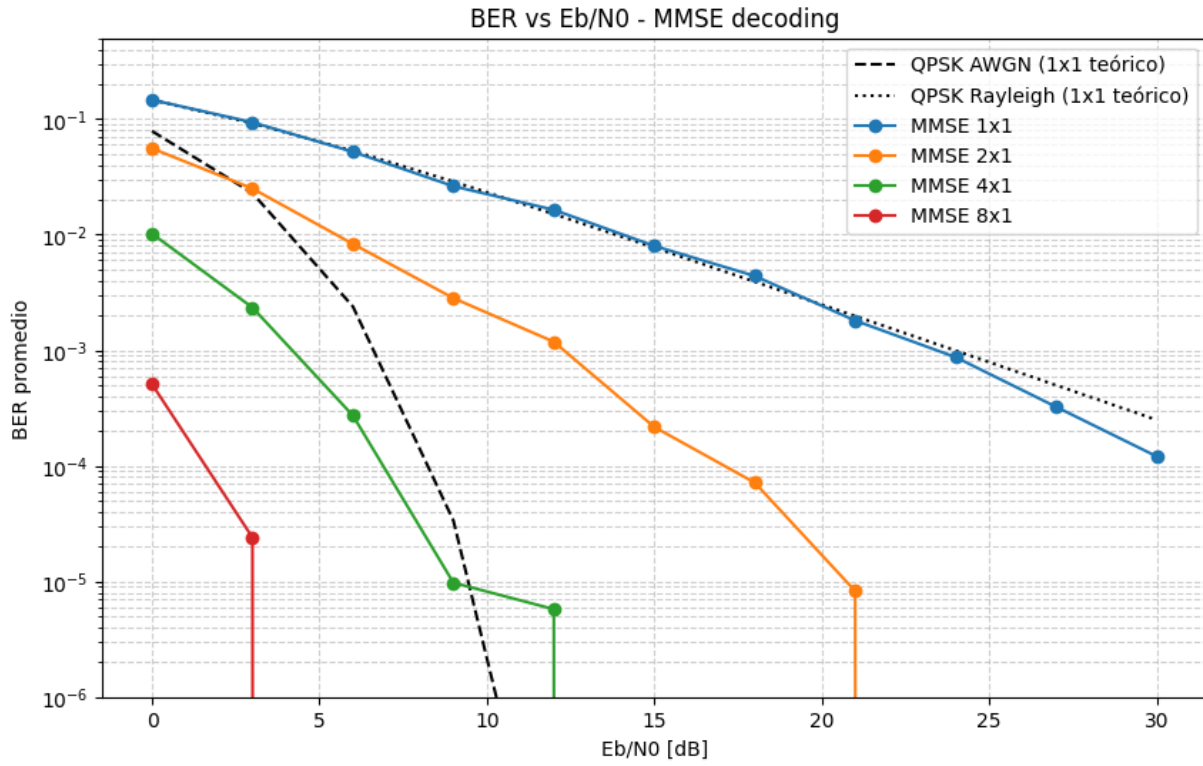


Figura 4: BER vs E_b/N_0 para configuraciones MMSE: 1x1, 2x1, 4x1 y 8x1.

MMSE entrega mejoras considerables en baja SNR respecto a ZF, especialmente visibles en 1x1 y 2x1. El desempeño debería converger al de ZF en alta SNR.

33.b) Simulación MMSE para configuraciones 1x1, 2x2, 4x4 y 8x8

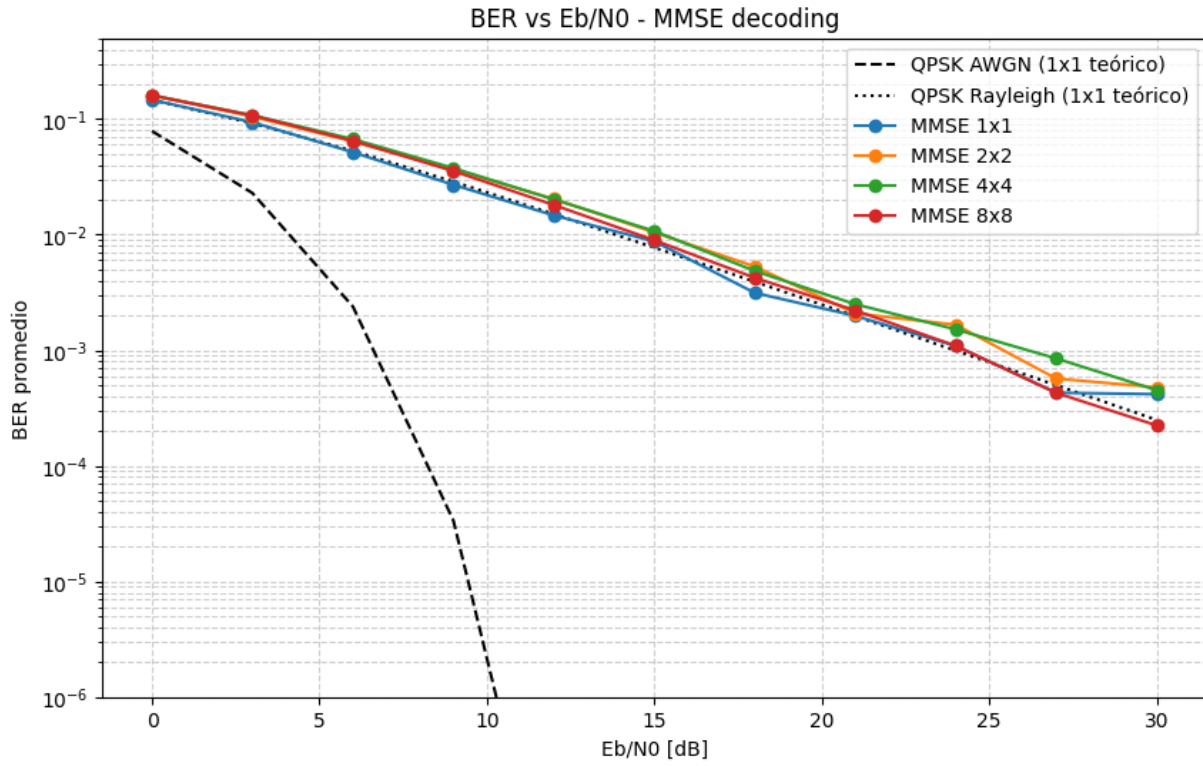


Figura 5: BER vs E_b/N_0 para configuraciones MMSE: 1x1, 2x2, 4x4 y 8x8.

En todos los casos MMSE supera a ZF en la región de baja relación señal-ruido. A medida que crece la matriz MIMO, la diferencia se reduce, dado que la diversidad compensa la amplificación del ruido propia del ZF.

33.c) Simulación MMSE para configuraciones 2x1, 4x2 y 8x4

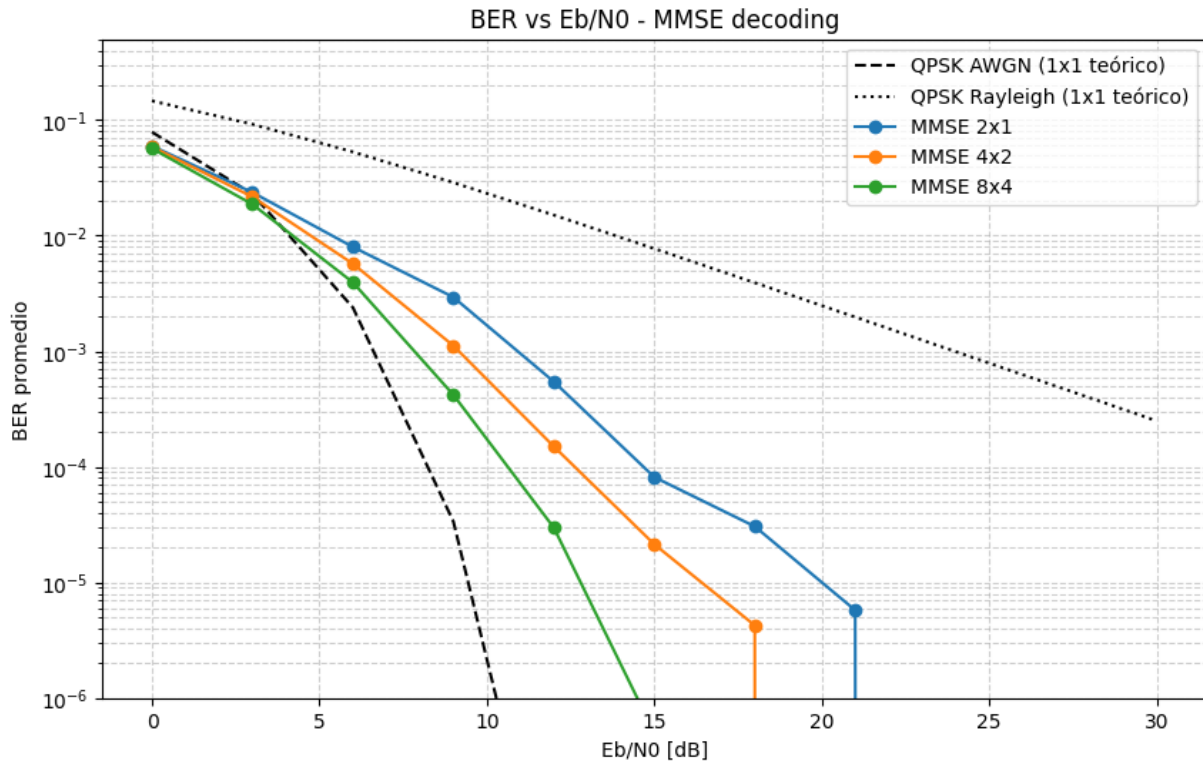


Figura 6: BER vs E_b/N_0 para configuraciones MMSE: 2×1, 4×2 y 8×4.

MMSE obtiene la mejor performance relativa en escenarios con más antenas receptoras que transmisoras, dado que puede explotar el exceso de dimensiones para reducir aún más el ruido.

Código utilizado

```
# Cell: Imports and environment detection
import os, time, math
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import erfc
from tqdm import tqdm
from multiprocessing import Pool, cpu_count

# Detect CuPy (GPU) availability
try:
    import cupy as cp
    gpu_available = True
    print('CuPy available. GPU mode possible.')
except Exception:
    cp = None
    gpu_available = False
    print('CuPy not available; GPU mode disabled.')

def get_xp(use_gpu=False):
    return cp if (use_gpu and gpu_available) else np
```

```

def qpsk_mod_bits_to_symbols(bits, xp):
    b0 = 1 - 2*bits[...,0]
    b1 = 1 - 2*bits[...,1]
    return (b0 + 1j*b1) / xp.sqrt(2)

def qpsk_demod_symbols_to_bits(symbols, xp):
    bits0 = (symbols.real < 0).astype(xp.int8)
    bits1 = (symbols.imag < 0).astype(xp.int8)
    return xp.stack([bits0, bits1], axis=-1)

def awgn_noise(shape, NO, xp):
    sigma = xp.sqrt(NO/2.0)
    return sigma * (xp.random.normal(size=shape) + 1j * xp.random.normal(size=shape))

def ber_qpsk_awgn(EbNO_lin):
    return 0.5 * erfc(np.sqrt(EbNO_lin))

def ber_qpsk_rayleigh(EbNO_lin):
    return 0.5 * (1 - np.sqrt(EbNO_lin / (1 + EbNO_lin)))

# CHECKPOINT QPSK rápido (CPU)
xp_test = np
bits_test = xp_test.random.randint(0,2,(50000,2))
s_test = qpsk_mod_bits_to_symbols(bits_test, xp_test)
print("\nCHECKPOINT QPSK: E[|s|^2] =", np.mean(np.abs(s_test)**2), "(=1)")

# Cell: Generadores batch
def gen_rayleigh_channel_batch(N, Nr, Nt, xp):
    real = xp.random.normal(size=(N, Nr, Nt))
    imag = xp.random.normal(size=(N, Nr, Nt))
    return (real + 1j*imag) / xp.sqrt(2.0)

def gen_qpsk_symbols_batch(N, Nt, Ns, Es_per_stream, xp):
    # Devuelve S_symbols shape (N, Nt, Ns) y bits (N,Nt,Ns,2)
    bits = xp.random.randint(0, 2, size=(N, Nt, Ns, 2))
    S = qpsk_mod_bits_to_symbols(bits, xp) # (N, Nt, Ns)
    S = S * xp.sqrt(Es_per_stream)
    return S, bits

# Cell: Transmisión + decodificación ZF en receptor
def transmit_and_zf_decode(H, S_streams, NO, Es, xp):
    """
    H: (N, Nr, Nt)
    S_streams: (N, Nt, Ns) -> símbolos transmitidos por cada antena tx
    Retorna: S_hat (N, Nt, Ns), S_streams (N,Nt,Ns)
    """
    N, Nr, Nt = H.shape
    _, Nt2, Ns = S_streams.shape
    assert Nt2 == Nt

    # Transmitir: X = S_streams (cada antena tx transmite su secuencia)
    X = S_streams # (N, Nt, Ns)
    Y = xp.einsum('nij, njs -> nis', H, X) # (N, Nr, Ns)
    Y += awgn_noise(Y.shape, NO, xp)

    # Decodificación ZF: W_zf = (H^H H)^{-1} H^H -> shape (N, Nt, Nr)
    H_H = H.conj().transpose(0, 2, 1) # (N, Nt, Nr)
    R = xp.matmul(H_H, H) # (N, Nt, Nt)

    # Chequeo de invertibilidad: usar pseudo-inversa si necesario
    s_hat = xp.zeros((N, Nt, Ns), dtype=complex)
    eps = 1e-12
    for i in range(N):
        try:
            Rinv = xp.linalg.inv(R[i])
        except Exception:

```

```

        # si no invertible, usar pseudo-inversa
        Rinv = xp.linalg.pinv(R[i])
        Wz = Rinv @ H_H[i] # (Nt, Nr)
        s_hat[i] = (Wz @ Y[i]) # (Nt, Ns)

    return s_hat, S_streams

# Cell: Transmisión + Decodificación MMSE (batch, CPU/GPU compatible)
def transmit_and_mmse_decode(H, S_streams, NO, Es, xp):
    """
    Decodificación MMSE en el receptor.
    H: (N, Nr, Nt)
    S_streams: (N, Nt, Ns) -> símbolos transmitidos por cada antena tx
    NO: ruido
    Es: energía total por símbolo (se reparte en Nt streams)
    xp: numpy o cupy
    Retorna: s_hat (N, Nt, Ns), S_streams (verdaderos)
    """
    N, Nr, Nt = H.shape
    _, Nt2, Ns = S_streams.shape
    assert Nt2 == Nt

    # Transmisión
    X = S_streams # (N,Nt,Ns)
    Y = xp.einsum('nij, njs -> nis', H, X) # (N,Nr,Ns)
    Y += awgn_noise(Y.shape, NO, xp)

    # Parámetro de regularización: NO / E_s_stream
    E_s_stream = Es / Nt
    reg = NO / (E_s_stream + 1e-18)

    s_hat = xp.zeros((N, Nt, Ns), dtype=complex)
    ident = None

    # Calculamos W_mmse por realización (puede vectorizarse si xp soporta batched solve)
    for i in range(N):
        Hi = H[i] # (Nr, Nt)
        H_H = Hi.conj().T # (Nt, Nr)
        A = H_H @ Hi # (Nt, Nt)
        if ident is None or ident.shape[0] != Nt:
            ident = xp.eye(Nt, dtype=A.dtype)
        A_reg = A + reg * ident
        # Resolver W = A_reg^{-1} H^H
        # Usar solve para mejores propiedades numéricas: solve(A_reg, H_H)
        try:
            Ainv_HH = xp.linalg.solve(A_reg, H_H) # (Nt, Nr)
        except Exception:
            # fallback pseudo-inversa si falla
            Ainv_HH = xp.linalg.pinv(A_reg) @ H_H
        W = Ainv_HH # (Nt, Nr)
        s_hat[i] = W @ Y[i] # (Nt, Ns)

    return s_hat, S_streams

# Cell: Worker CPU para ZF (para map/Pool)
def simulate_for_EbNO_zf_cpu(args):
    Nr, Nt, EbNO_lin, Ns, Nreal, seed = args
    xp = np
    rng = np.random.default_rng(seed)
    NO = 1.0
    Es = EbNO_lin * (2 * NO) # convención: Es = EbNO * (2*NO)
    Es_per_stream = Es / Nt # se reparte entre antenas tx
    chunk = 200
    total_errors = 0
    total_bits = 0

```



```

for start in range(0, Nreal, chunk):
    thisN = min(chunk, Nreal - start)
    H = gen_rayleigh_channel_batch(thisN, Nr, Nt, xp)
    S_streams, bits_streams = gen_qpsk_symbols_batch(thisN, Nt, Ns, Es_per_stream, xp)
    s_hat, s_true = transmit_and_zf_decode(H, S_streams, NO, Es, xp)

    # Demodular y contar errores (replicar bits shape compat)
    bits_hat = qpsk_demod_symbols_to_bits(s_hat, xp) # (N, Nt, Ns, 2)
    total_errors += int(np.sum(bits_hat != bits_streams))
    total_bits += bits_hat.size

Pb = total_errors / total_bits
return EbNO_lin, Pb

# GPU worker (similar, en batch si Cupy está disponible)
def simulate_for_EbNO_zf_gpu(Nr, Nt, EbNO_lin, Ns, Nreal, seed):
    xp = cp
    rng = xp.random.default_rng(seed)
    NO = 1.0
    Es = EbNO_lin * (2 * NO)
    Es_per_stream = Es / Nt

    H = gen_rayleigh_channel_batch(Nreal, Nr, Nt, xp)
    S_streams, bits_streams = gen_qpsk_symbols_batch(Nreal, Nt, Ns, Es_per_stream, xp)
    s_hat, s_true = transmit_and_zf_decode(H, S_streams, NO, Es, xp)

    bits_hat = qpsk_demod_symbols_to_bits(s_hat, xp)
    errors = int((bits_hat != bits_streams).sum().get())
    total_bits = bits_hat.size
    Pb = errors / total_bits
    return EbNO_lin, Pb

# Cell: Workers MMSE (CPU / GPU)
def simulate_for_EbNO_mmse_cpu(args):
    Nr, Nt, EbNO_lin, Ns, Nreal, seed = args
    xp = np
    rng = np.random.default_rng(seed)
    NO = 1.0
    Es = EbNO_lin * (2 * NO)
    Es_per_stream = Es / Nt
    chunk = 200
    total_errors = 0
    total_bits = 0

    for start in range(0, Nreal, chunk):
        thisN = min(chunk, Nreal - start)
        H = gen_rayleigh_channel_batch(thisN, Nr, Nt, xp)
        S_streams, bits_streams = gen_qpsk_symbols_batch(thisN, Nt, Ns, Es_per_stream, xp)
        s_hat, s_true = transmit_and_mmse_decode(H, S_streams, NO, Es, xp)

        bits_hat = qpsk_demod_symbols_to_bits(s_hat, xp)
        total_errors += int(np.sum(bits_hat != bits_streams))
        total_bits += bits_hat.size

    Pb = total_errors / total_bits
    return EbNO_lin, Pb

def simulate_for_EbNO_mmse_gpu(Nr, Nt, EbNO_lin, Ns, Nreal, seed):
    xp = cp
    rng = xp.random.default_rng(seed)
    NO = 1.0
    Es = EbNO_lin * (2 * NO)
    Es_per_stream = Es / Nt

    H = gen_rayleigh_channel_batch(Nreal, Nr, Nt, xp)
    S_streams, bits_streams = gen_qpsk_symbols_batch(Nreal, Nt, Ns, Es_per_stream, xp)
    s_hat, s_true = transmit_and_mmse_decode(H, S_streams, NO, Es, xp)

```

```

bits_hat = qpsk_demod_symbols_to_bits(s_hat, xp)
errors = int((bits_hat != bits_streams).sum().get())
total_bits = bits_hat.size
Pb = errors / total_bits
return EbNO_lin, Pb

# Cell: Orquestador ZF decoding (CPU/GPU)
def run_zf_decoding_simulation(configs, EbNO_dB, Ns=1000, Nreal=10000, use_gpu=False, nprocs=None):
    EbNO_lin_list = 10**(EbNO_dB/10)
    results = {}
    if use_gpu and gpu_available:
        for (Nr, Nt) in configs:
            print(f'GPU mode: running {Nr}x{Nt} with ZF decoding')
            Pb_list = []
            for EbNO_lin in EbNO_lin_list:
                _, Pb = simulate_for_EbNO_zf_gpu(Nr, Nt, EbNO_lin, Ns, Nreal, seed=12345)
                Pb_list.append(Pb)
            results[(Nr, Nt)] = np.array(Pb_list)
    else:
        if nprocs is None:
            nprocs = max(1, cpu_count()-1)
        pool = Pool(processes=nprocs)
        try:
            for (Nr, Nt) in configs:
                print(f'CPU multi: running {Nr}x{Nt} (nprocs={nprocs}) with ZF decoding')
                args = [(Nr, Nt, EbNO_lin, Ns, max(1, Nreal//nprocs), 1000 + i)
                        for i, EbNO_lin in enumerate(EbNO_lin_list)]
                res = pool.map(simulate_for_EbNO_zf_cpu, args)
                # ordenar por EbNO
                res_sorted = sorted(res, key=lambda x: list(EbNO_lin_list).index(x[0]))
                Pb_list = [r[1] for r in res_sorted]
                results[(Nr, Nt)] = np.array(Pb_list)
        finally:
            pool.close()
            pool.join()
    return results

# Cell: Orquestador MMSE (CPU/GPU)
def run_mmse_decoding_simulation(configs, EbNO_dB, Ns=1000, Nreal=10000, use_gpu=False, nprocs=None):
    EbNO_lin_list = 10**(EbNO_dB/10)
    results = {}
    if use_gpu and gpu_available:
        for (Nr, Nt) in configs:
            print(f'GPU: running MMSE {Nr}x{Nt}')
            Pb_list = []
            for EbNO_lin in EbNO_lin_list:
                _, Pb = simulate_for_EbNO_mmse_gpu(Nr, Nt, EbNO_lin, Ns, Nreal, seed=12345)
                Pb_list.append(Pb)
            results[(Nr, Nt)] = np.array(Pb_list)
    else:
        if nprocs is None:
            nprocs = max(1, cpu_count()-1)
        pool = Pool(processes=nprocs)
        try:
            for (Nr, Nt) in configs:
                print(f'CPU multi: running MMSE {Nr}x{Nt} (nprocs={nprocs})')
                args = [(Nr, Nt, EbNO_lin, Ns, max(1, Nreal//nprocs), 2000 + i)
                        for i, EbNO_lin in enumerate(EbNO_lin_list)]
                res = pool.map(simulate_for_EbNO_mmse_cpu, args)
                res_sorted = sorted(res, key=lambda x: list(EbNO_lin_list).index(x[0]))
                Pb_list = [r[1] for r in res_sorted]
                results[(Nr, Nt)] = np.array(Pb_list)
        finally:
            pool.close()
            pool.join()
    return results

```

```

# Cell: Parámetros y ejecución ZF decoding
EbNO_dB = np.arange(0, 31, 3)
configs = [(1,1), (2,1), (4,1), (8,1)] # (Nr, Nt)
Ns = 1000
Nreal = 10000
use_gpu = gpu_available
print("Modo:", "GPU" if use_gpu else "CPU")

start = time.time()
results_zf = run_zf_decoding_simulation(configs, EbNO_dB, Ns=Ns, Nreal=Nreal, use_gpu=use_gpu, nprocs=4)
end = time.time()
print("Tiempo total (s):", end - start)

# Cell: Plot resultados ZF decoding
plt.figure(figsize=(10,6))
EbNO_lin = 10**(EbNO_dB/10)
plt.semilogy(EbNO_dB, ber_qpsk_awgn(EbNO_lin), 'k--', label='QPSK AWGN (1x1 teórico)')
plt.semilogy(EbNO_dB, ber_qpsk_rayleigh(EbNO_lin), 'k:', label='QPSK Rayleigh (1x1 teórico)')
for (Nr, Nt), Pb in results_zf.items():
    plt.semilogy(EbNO_dB, Pb, 'o-', label=f'ZF decoding {Nr}x{Nt}')
plt.title('BER vs Eb/NO - ZF decoding')
plt.xlabel('Eb/NO [dB]')
plt.ylabel('BER promedio')
plt.grid(True, which='both', ls='--', alpha=0.6)
plt.ylim(1e-6, 0.5)
plt.legend()
plt.show()

# Cell: Ejecución MMSE
EbNO_dB = np.arange(0, 31, 3)
configs_test = [(1,1), (2,1), (4,1), (8,1)]
Ns = 1000
Nreal = 2000 # reduce para pruebas rápidas; poner 10000 en corrida final
use_gpu = gpu_available

print("Running MMSE decoding sims (may take time)... mode:", "GPU" if use_gpu else "CPU")
t0 = time.time()
results_mmse = run_mmse_decoding_simulation(configs_test, EbNO_dB, Ns=Ns, Nreal=Nreal, use_gpu=use_gpu,
↪ nprocs=4)
t1 = time.time()
print("Elapsed (s):", t1-t0)

# Cell: Plot comparación (MMSE)
plt.figure(figsize=(10,6))
EbNO_lin = 10**(EbNO_dB/10)
plt.semilogy(EbNO_dB, ber_qpsk_awgn(EbNO_lin), 'k--', label='QPSK AWGN (1x1 teórico)')
plt.semilogy(EbNO_dB, ber_qpsk_rayleigh(EbNO_lin), 'k:', label='QPSK Rayleigh (1x1 teórico)')
for (Nr, Nt), Pb in results_mmse.items():
    plt.semilogy(EbNO_dB, Pb, 'o-', label=f'MMSE {Nr}x{Nt}')
# si tienes ZF results, plotealas también:
# for (Nr,Nt), Pb in results_zf.items(): plt.semilogy(EbNO_dB, Pb, 's--', label=f'ZF {Nr}x{Nt}')

plt.title('BER vs Eb/NO - MMSE decoding')
plt.xlabel('Eb/NO [dB]')
plt.ylabel('BER promedio')
plt.grid(True, which='both', ls='--', alpha=0.6)
plt.ylim(1e-6, 0.5)
plt.legend()
plt.show()

```